

ОБНАРУЖЕНИЕ И КЛАССИФИКАЦИЯ КИБЕРАТАК НА ОСНОВЕ КОМПЛЕКСНОГО АНАЛИЗА СЕТЕВОГО ТРАФИКА И ЖУРНАЛОВ СОБЫТИЙ: АЛГОРИТМИЧЕСКАЯ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

А.П. Васильченко, Е.А. Попова, Н.П. Жуков, А.Е. Дешина

В статье рассматривается разработка и программная реализация системы автоматизированного обнаружения и классификации кибератак, основанной на независимом и параллельном анализе сетевого трафика и журналов регистрации событий. Предложен архитектурный подход, предусматривающий разделение потоков обработки данных и последующее агрегирование результатов в терминах классификатора атак CAPEC. Реализованы модули сбора и агрегации сетевых потоков, обработки событийных данных и вспомогательной телеметрии, предобработки выявленных признаков и формирования итоговых решений. Проведена демонстрация работы системы в условиях локальной сети, подтвердившая корректность функционирования модулей и способность выявлять атакующие сценарии, проявляющиеся на различных уровнях информационной системы. Полученные результаты могут быть использованы при создании и развитии систем мониторинга и реагирования на инциденты информационной безопасности.

Ключевые слова: идентификация кибератак, классификация кибератак, сетевой трафик, журналы регистрации событий, CAPEC.

Введение

Современные компьютерные сети функционируют в условиях устойчивого роста сложности и разнообразия киберугроз. Атакующие сценарии становятся более распределёнными, многоэтапными и скрытыми, что существенно усложняет их своевременное выявление и требует применения решений, способных анализировать проявления атак как на сетевом уровне, так и на уровне конечных узлов [1]. В ответ на это в практике информационной безопасности сформировался широкий спектр средств обнаружения атак, однако их функциональные возможности во многом остаются ограниченными.

Анализ существующих решений показывает, что современные системы обнаружения и реагирования на кибератаки можно условно разделить на несколько основных классов.

Сетевые системы обнаружения вторжений (Network IDS/IPS) ориентированы на анализ сетевого трафика и выявление атакующих воздействий на уровне протоколов и сетевых соединений. К данному классу относятся следующие решения:

- Snort [2];
- Suricata [3];
- Zeek IDS [4];
- Cisco Secure IDS [5];
- Fortinet FortiGate IPS [6].

Указанные системы демонстрируют высокую эффективность при обнаружении сетевых сканирований, эксплуатации уязвимостей, атак отказа в обслуживании и аномального сетевого поведения [2–4]. Вместе с тем их функциональные возможности ограничиваются анализом сетевого уровня, вследствие чего атаки, проявляющиеся преимущественно в событиях операционных систем и прикладных сервисов, могут оставаться необнаруженными.

Хостовые средства обнаружения и реагирования (HIDS/EDR) ориентированы на анализ событий, происходящих внутри операционных систем и приложений. К данной группе относятся:

- Wazuh [7];
- OSSEC [8];
- Kaspersky Endpoint Detection and Response [9];
- CrowdStrike Falcon [10];
- Microsoft Defender for Endpoint [11].

Данные решения позволяют фиксировать запуск и завершение процессов, изменения конфигурации, попытки несанкционированного доступа и признаки вредоносной активности на конечных узлах [5–7]. Однако такие системы, как правило, не осуществляют полноценный анализ сетевого трафика, что затрудняет выявление атак, начинающихся на сетевом уровне и развивающихся внутри инфраструктуры.

Комплексные платформы управления событиями безопасности (SIEM/SOC) обеспечивают сбор, хранение и корреляцию данных из различных источников, включая сетевой трафик, журналы регистрации событий и телеметрию средств защиты. К числу таких решений относятся:

- Splunk Enterprise Security [12];
- IBM QRadar [13];
- ArcSight [14];
- Elastic Security [15];
- Graylog Security [16];
- Positive Technologies PT NAD [17].

Несмотря на широкий охват источников данных, в большинстве случаев данные платформы опираются на заранее заданные правила корреляции и сигнатурные механизмы, а полноценная автоматизированная классификация атакующих сценариев в соответствии с формализованными классификаторами угроз реализована ограниченно.

Обобщение сильных и слабых сторон существующих решений позволяет выделить ключевое противоречие, определяющее актуальность настоящего исследования:

- между необходимостью параллельного и независимого анализа сетевого трафика и журналов регистрации событий и архитектурной ориентацией большинства существующих систем на обработку преимущественно одного типа данных;

- между потребностью в автоматизированной классификации атакующих сценариев в соответствии с формализованными классификаторами угроз и ограниченными возможностями существующих систем по интерпретации и согласованию результатов анализа разнородных источников данных.

Выявленные противоречия указывают на необходимость разработки архитектурного подхода, обеспечивающего независимый и параллельный анализ сетевого трафика и журналов событий с последующей агрегацией результатов и интерпретацией атакующих сценариев в рамках единой системы обнаружения и классификации.

Целью настоящей статьи является разработка алгоритмической и программной архитектуры системы автоматизированного обнаружения и классификации кибератак, обеспечивающей независимый сбор, обработку и анализ сетевого трафика и журналов регистрации событий с последующей агрегацией результатов и интерпретацией атакующих сценариев.

Для достижения поставленной цели в статье решаются следующие задачи:

- 1) разработка архитектуры и алгоритмов модулей независимого сбора и предварительной обработки сетевого трафика и журналов регистрации событий;
- 2) разработка алгоритмов интерпретации и согласования результатов анализа разнородных источников данных в рамках единой системы принятия решений.

Архитектура системы автоматизированного обнаружения и классификации кибератак

Архитектура системы автоматизированного обнаружения и классификации кибератак ориентирована на практическую реализацию алгоритмов независимой обработки разнородных источников данных и построена с учётом необходимости параллельного анализа сетевого трафика и журналов регистрации событий. Такой подход обусловлен различиями в характере проявления атакующих сценариев, а также ограничениями, присущими системам, использующим только один источник информации.

Для наглядного представления общей структуры системы автоматизированного обнаружения и классификации кибератак на рис. 1 приведена архитектурная схема, отражающая основные функциональные подсистемы и их взаимодействие. Схема иллюстрирует принцип независимой

обработки сетевого трафика и журналов агрегацию результатов анализа в модуле регистрации событий, а также последующую принятия решений.

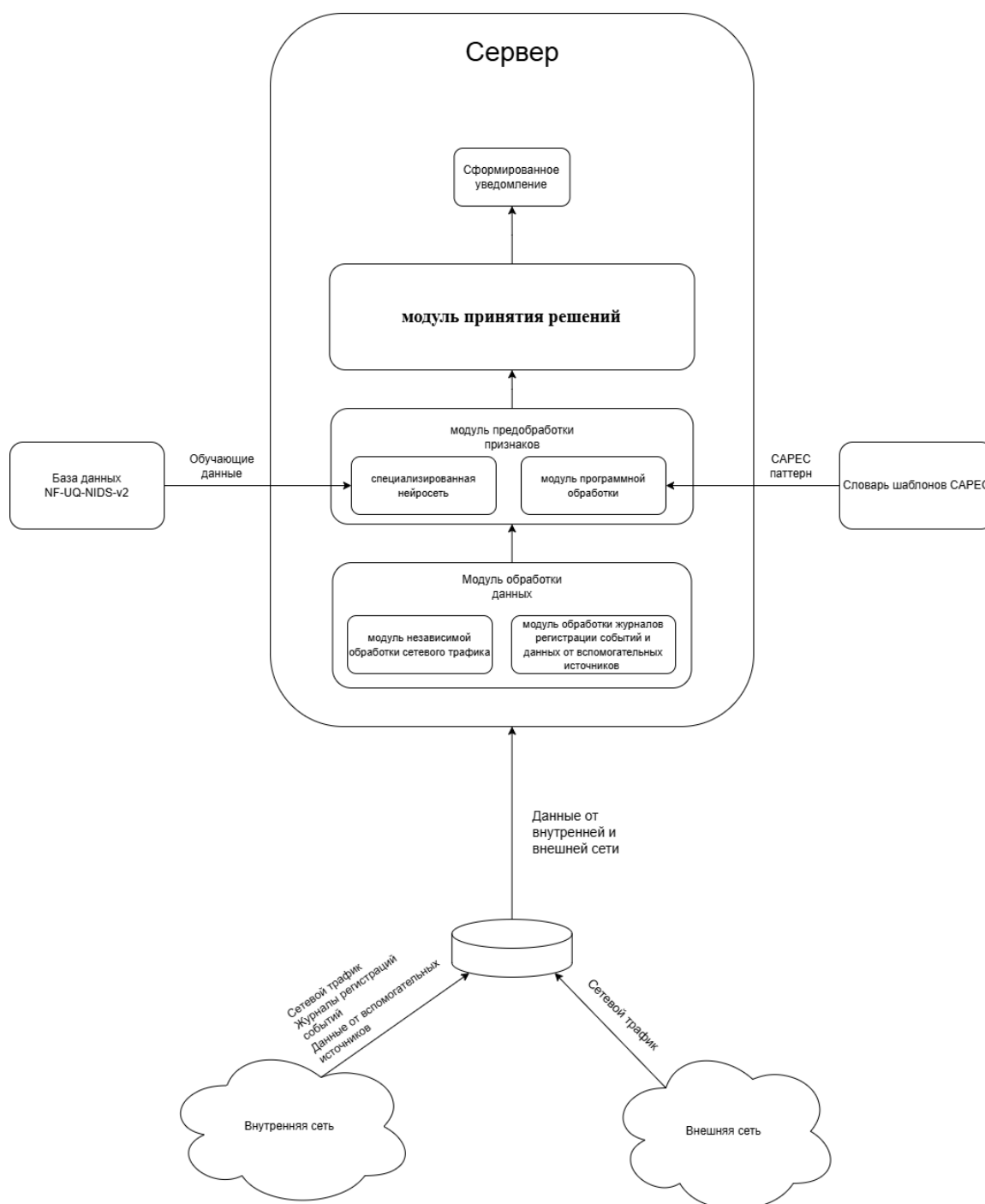


Рис. 1. Архитектура системы автоматизированного обнаружения и классификации кибератак на основе независимой обработки сетевого трафика и журналов регистрации событий

Каждый функциональный модуль реализует строго определённый этап обработки данных, а взаимодействие между модулями осуществляется через формализованные интерфейсы. Это обеспечивает масштабируемость архитектуры, упрощает сопровождение программного обеспечения и позволяет

расширять набор детектируемых шаблонов атак без нарушения существующей логики работы системы.

Структурно система включает четыре ключевые подсистемы.

Первая подсистема — модуль обработки сетевого трафика. Он отвечает за приём сетевых данных, их фильтрацию, агрегацию и

нормализацию, а также за формирование признаков сетевых потоков, отражающих статистические и поведенческие характеристики сетевых взаимодействий. Результатом работы модуля является структурированное представление сетевого трафика, пригодное для последующего анализа. Подготовленные данные передаются в предварительно обученную классификационную модель, предназначенную для автоматизированной идентификации атакующих сценариев, обладающих устойчивыми сетевыми проявлениями. Архитектура и процесс обучения модели рассматриваются в отдельной работе и в рамках настоящей статьи не детализируются.

Вторая подсистема — модуль обработки журналов регистрации событий и вспомогательных источников данных. Данный модуль реализует программные алгоритмы анализа событий, фиксируемых на уровне операционных систем, прикладных сервисов и сетевой телеметрии. В качестве входных данных используются системные и прикладные журналы, данные сетевых протоколов, а также события и аномалии, формируемые средствами мониторинга и обнаружения вторжений. Подсистема предназначена для выявления атакующих сценариев, признаки которых слабо выражены или отсутствуют в сетевом трафике и проявляются преимущественно в событийных источниках. Тем самым модуль обеспечивает расширение перечня обнаруживаемых шаблонов атак за счёт сценариев, недоступных для выявления исключительно по сетевым признакам.

Третья подсистема — модуль согласования и предварительной обработки данных, обеспечивающий корректное объединение информации, поступающей из двух независимых потоков анализа. На данном этапе выполняется проверка целостности входных данных, согласование временных меток, унификация форматов представления и формирование согласованных структур, используемых в последующих этапах обработки. Работа данного модуля позволяет устранить различия между сетевыми и событийными источниками и обеспечить корректное

взаимодействие подсистем в рамках единой архитектуры.

Четвёртая подсистема — модуль принятия решений, осуществляющий агрегацию результатов, полученных от классификационной модели и программных алгоритмов анализа событийных данных. Модуль формирует итоговый вывод о наличии и типе атакующего сценария, выполняет его сопоставление с соответствующими шаблонами CAPEC и обеспечивает регистрацию выявленных инцидентов, а также формирование уведомлений для оператора системы безопасности [18].

Предложенная архитектура обеспечивает независимую и параллельную обработку сетевого трафика и журналов регистрации событий, что позволяет повысить полноту обнаружения атакующих сценариев и обеспечить практическую применимость системы в условиях реальных корпоративных и распределённых сетевых инфраструктур.

Модуль независимой обработки сетевого трафика

Модуль независимой обработки сетевого трафика предназначен для автоматизированного сбора сетевых данных и их преобразования в структурированное представление, пригодное для дальнейшего анализа. Основной задачей модуля является переход от обработки отдельных сетевых пакетов к формированию агрегированных объектов — сетевых потоков, описывающих параметры и характер наблюдаемого сетевого взаимодействия. На данном этапе не выполняется классификация атак и не принимаются решения о наличии инцидентов безопасности; модуль реализует исключительно функции сбора, первичной обработки и структурирования сетевой информации.

Общая логика функционирования модуля представлена на рис. 2 и отражает последовательность ключевых этапов обработки данных, начиная с захвата сетевых пакетов и заканчивая формированием унифицированного потокового представления. Работа модуля организована в виде циклического процесса, что позволяет осуществлять анализ сетевого трафика в

режиме, близком к реальному времени, с использованием фиксированного временного окна обработки.



Рис. 2. Блок-схема алгоритма работы модуля обработки сетевого трафика

Работа модуля начинается с этапа инициализации, в рамках которого задаются параметры функционирования системы: выбирается сетевой интерфейс для захвата пакетов, определяется длительность временного окна анализа и настраивается BPF-фильтр, ограничивающий объём анализируемого трафика. Применение фильтрации на уровне захвата позволяет снизить вычислительную нагрузку и исключить обработку пакетов, не относящихся к рассматриваемым сценариям. Пример задания параметров и конфигурации режима работы модуля приведён на рис. 3.

После завершения инициализации модуль переходит в режим непрерывного захвата данных. В пределах каждого временного окна осуществляется сбор сетевых пакетов с выбранного интерфейса. Такой подход позволяет отказаться от накопления больших объёмов данных и обрабатывать трафик порциями фиксированного размера, обеспечивая предсказуемость и устойчивость работы системы. Реализация циклического захвата пакетов и обработка ситуаций отсутствия данных в пределах окна показаны на рис. 4.

Первичная фильтрация и проверка корректности пакетов выполняются на двух уровнях. Часть трафика отсекается ещё на этапе захвата за счёт применения BPF-фильтра, после чего выполняется программная проверка структуры пакетов. На данном этапе контролируется наличие корректных IP-адресов источника и назначения, допустимость извлекаемых полей и корректность типов данных. Такой механизм обеспечивает устойчивость работы модуля в условиях реального сетевого трафика, содержащего повреждённые или нестандартные пакеты. Пример соответствующей логики фильтрации приведён на рис. 5.


```

420 parser = argparse.ArgumentParser(
421     description="Online DPI (tshark) → model inference + анализ логов/SSH"
422 )
423 parser.add_argument("--iface", default=None, help="Интерфейс, например eth0")
424 parser.add_argument("--interval", type=float, default=2.0, help="Длительность окна захвата, сек")
425 parser.add_argument("--artifacts_dir", required=True, help="Путь к model.keras, preprocessor.joblib, encoders.joblib")
426 parser.add_argument("--devices", default="auto", help="GPU: auto|cpu|0,1,...")
427 parser.add_argument("--amp", type=int, default=1, help="Mixed precision (1/0)")
428 parser.add_argument("--bpf", default=None, help="Доп. BPF фильтр, напр. 'tcp or udp'")
429 parser.add_argument("--logs-dir", default="", help="Локальная папка для логов (web/auth/app/ids/zeek/firewall/system)")
430 parser.add_argument(
431     "--ssh-host",
432     action="append",
433     default=[],
434     help="Удалённый хост в формате user@host или user@host:password (для сбора логов по SSH)",
435 )
436 args = parser.parse_args()

```

Рис. 3. Задание параметров функционирования модуля обработки сетевого трафика

```

492 print("Старт DPI-инференса. Ctrl+C – остановка.")
493 while True:
494     t0 = time.time()
495     cap = pyshark.LiveCapture(interface=args.iface, bpf_filter=args.bpf)
496     try:
497         cap.sniff(timeout=args.interval)
498     except Exception:
499         pass
500
501     if not cap or len(cap) == 0:
502         print("...нет пакетов за окно.")
503         try:
504             cap.close()
505         except Exception:
506             pass
507         continue

```

Рис. 4. Реализация циклического захвата сетевых пакетов в пределах временного окна

```

252 proto, sport, dport = get_l4_and_ports(pkt)
253 key = (src, dst, proto, dport)
254 rec = self.by_flow[key]
255
256 if rec["_first_ts"] is None:
257     rec["_first_ts"] = ts
258     rec["_initiator"] = (src, sport)
259     rec["_last_ts"] = ts
260

```

Рис. 5. Первичная проверка и фильтрация сетевых пакетов перед агрегацией

Ключевым этапом функционирования модуля является агрегация сетевых пакетов в сетевые потоки. Каждый корректный пакет передаётся в агрегатор, который группирует пакеты по совокупности признаков соединения, включая IP-адреса, номер транспортного протокола и порты. В результате вместо набора разрозненных пакетов формируется единый объект, описывающий сетевое взаимодействие между узлами. Процесс агрегации и

формирование таблицы потоков представлены на рис. 6.

```

509 agg = FlowAgg()
510 for pkt in cap._packets:
511     try:
512         agg.add(pkt)
513     except Exception:
514         continue

```

Рис. 6. Агрегация сетевых пакетов в структурированные сетевые потоки

В ходе агрегации из пакетов извлекаются характеристики сетевого и транспортного уровней (L3/L4), такие как адреса источника и назначения, тип протокола, номера портов, длина пакета и значения TTL. Одновременно накапливаются статистические характеристики потоков, включая количество пакетов и байт в каждом направлении,

распределение размеров пакетов, минимальные и максимальные значения параметров, а также длительность потока. Примеры извлечения базовых характеристик пакетов и накопления статистик потоков приведены на рисунках 7 и 8, а расчёт производных метрик и временных характеристик — на рис. 9.

```

109 ∨ def get_ip_addrs(pkt) -> Tuple[str,str]:
110 ∨     try:
111 ∨         if hasattr(pkt, "ip"):
112 ∨             return pkt.ip.src, pkt.ip.dst
113 ∨         if hasattr(pkt, "ipv6"):
114 ∨             return pkt.ipv6.src, pkt.ipv6.dst
115 ∨     except Exception:
116 ∨         pass
117 ∨     return None, None
118
119 ∨ def get_ip_ttl(pkt) -> int:
120 ∨     try:
121 ∨         if hasattr(pkt, "ip") and hasattr(pkt.ip, "ttl"):
122 ∨             return to_int_safe(pkt.ip.ttl, 0)
123 ∨         if hasattr(pkt, "ipv6") and hasattr(pkt.ipv6, "hlim"):
124 ∨             return to_int_safe(pkt.ipv6.hlim, 0)
125 ∨     except Exception:
126 ∨         pass
127 ∨     return 0
128

```

Рис. 7. Извлечение базовых характеристик пакета сетевого и транспортного уровней

```

129 ∨ def get_l4_and_ports(pkt) -> Tuple[int,int,int]:
130 ∨     try:
131 ∨         if hasattr(pkt, "tcp"):
132 ∨             return 6, to_int_safe(pkt.tcp.srcport), to_int_safe(pkt.tcp.dstport)
133 ∨         if hasattr(pkt, "udp"):
134 ∨             return 17, to_int_safe(pkt.udp.srcport), to_int_safe(pkt.udp.dstport)
135 ∨         if hasattr(pkt, "icmp"):
136 ∨             return 1, 0, 0
137 ∨     except Exception:
138 ∨         pass
139 ∨     return 0, 0, 0
140
141 ∨ def tcp_flags_mask(pkt) -> int:
142 ∨     f = 0
143 ∨     try:
144 ∨         if not hasattr(pkt, "tcp"): return 0
145 ∨         bits = {
146 ∨             "fin": 0x01, "syn": 0x02, "rst": 0x04, "psh": 0x08,
147 ∨             "ack": 0x10, "urg": 0x20, "ece": 0x40, "cwr": 0x80
148 ∨         }
149 ∨         for name, bit in bits.items():
150 ∨             attr = f"flags_{name}"
151 ∨             if hasattr(pkt.tcp, attr) and str(getattr(pkt.tcp, attr)) == "1":
152 ∨                 f |= bit
153 ∨     except Exception:
154 ∨         pass
155 ∨     return f
156
157 ∨ def tcp_win(pkt) -> int:
158 ∨     try:
159 ∨         if hasattr(pkt, "tcp") and hasattr(pkt.tcp, "window_size_value"):
160 ∨             return to_int_safe(pkt.tcp.window_size_value, 0)
161 ∨     except Exception:
162 ∨         pass
163 ∨     return 0
164
165 ∨ def icmp_type(pkt) -> int:
166 ∨     try:
167 ∨         if hasattr(pkt, "icmp") and hasattr(pkt.icmp, "type"):
168 ∨             return to_int_safe(pkt.icmp.type, 0)
169 ∨     except Exception:
170 ∨         pass
171 ∨     return 0

```

Рис. 8. Накопление статистических характеристик сетевого потока

```

242  def add(self, pkt):
243      try:
244          ts = float(getattr(pkt, "sniff_time").timestamp())
245      except Exception:
246          ts = time.time()
247
248      src, dst = get_ip_addrs(pkt)
249      if not src or not dst:
250          return
251
252      proto, sport, dport = get_l4_and_ports(pkt)
253      key = (src, dst, proto, dport)
254      rec = self.by_flow[key]
255
256      if rec["_first_ts"] is None:
257          rec["_first_ts"] = ts
258          rec["_initiator"] = (src, sport)
259      rec["_last_ts"] = ts
260
261      rec["L4_DST_PORT"] = dport
262      rec["PROTOCOL"] = proto
263      if np.isnan(rec["L7_PROTO"]):
264          rec["L7_PROTO"] = l7_from_pyshark(pkt, sport, dport)
265
266      blen = get_len(pkt)
267      ttl = get_ip_ttl(pkt)
268      if ttl > 0:
269          rec["MIN_TTL"] = min(rec["MIN_TTL"], ttl)
270          rec["MAX_TTL"] = max(rec["MAX_TTL"], ttl)
271      rec["LONGEST_FLOW_PKT"] = max(rec["LONGEST_FLOW_PKT"], blen)
272      if blen > 0:
273          rec["SHORTEST_FLOW_PKT"] = min(rec["SHORTEST_FLOW_PKT"], blen)
274          rec["MIN_IP_PKT_LEN"] = min(rec["MIN_IP_PKT_LEN"], blen)
275          rec["MAX_IP_PKT_LEN"] = max(rec["MAX_IP_PKT_LEN"], blen)
276
277      if blen <= 128: rec["NUM_PKTS_UP_TO_128_BYTES"] += 1
278      elif blen <= 256: rec["NUM_PKTS_128_TO_256_BYTES"] += 1
279      elif blen <= 512: rec["NUM_PKTS_256_TO_512_BYTES"] += 1
280      elif blen <= 1024: rec["NUM_PKTS_512_TO_1024_BYTES"] += 1
281      elif blen <= 1514: rec["NUM_PKTS_1024_TO_1514_BYTES"] += 1
282
283      out_dir = is_local(src) or not is_local(dst)
284      in_dir = is_local(dst)
285
286      if in_dir:
287          rec["IN_BYTES"] += blen
288          rec["IN_PKTS"] += 1
289          if rec["_in_first_ts"] is None: rec["_in_first_ts"] = ts
290          rec["_in_last_ts"] = ts

```

Рис. 9. Расчёт производных метрик и временных характеристик потоков

Дополнительно модуль выполняет идентификацию протоколов прикладного уровня (L7). Определение прикладного протокола осуществляется на основе анализа структуры пакетов и информации о протокольных слоях, предоставляемой анализатором пакетов. В случае отсутствия явных признаков применяется резервный

механизм, основанный на известных соответствиях портов прикладным протоколам. Таблица таких соответствий приведена на рис. 10, алгоритм идентификации L7-протокола — на рис. 11, а пример использования полученной информации при агрегации потоков — на рис. 12.

```

67  L7_MAP = {
68      "HTTP": 7, "TLS": 8, "SSL": 8,
69      "DNS": 2, "FTP": 10, "SSH": 11, "SMTP": 12, "POP": 13, "POP3": 13, "IMAP": 14,
70      "BOOTP": 15, "DHCP": 15, "NTP": 17,
71      "MYSQL": 18, "PGSQL": 19, "REDIS": 20, "MONGODB": 21,
72  }

```

Рис. 10. Таблица соответствий прикладных протоколов (L7)


```

198 def l7_from_pyshark(pkt, sport, dport) -> int:
199     try:
200         hl = getattr(pkt, "highest_layer", None)
201         if hl and hl.upper() in L7_MAP:
202             return L7_MAP[hl.upper()]
203     except Exception:
204         pass
205     for name, code in L7_MAP.items():
206         if hasattr(pkt, name.lower()):
207             return code
208     WELL_KNOWN = {80:7, 443:8, 53:2, 21:10, 22:11, 25:12, 110:13, 143:14, 67:15, 68:15, 123:17, 3306:18, 5432:19, 6379:20, 27017:21}
209     return WELL_KNOWN.get(dport, WELL_KNOWN.get(sport, 0))
210

```

Рис. 11. Алгоритм идентификации протокола прикладного уровня

```

268         if ttl > 0:
269             rec["MIN_TTL"] = min(rec["MIN_TTL"], ttl)
270             rec["MAX_TTL"] = max(rec["MAX_TTL"], ttl)
271         rec["LONGEST_FLOW_PKT"] = max(rec["LONGEST_FLOW_PKT"], blen)
272         if blen > 0:
273             rec["SHORTEST_FLOW_PKT"] = min(rec["SHORTEST_FLOW_PKT"], blen)
274             rec["MIN_IP_PKT_LEN"] = min(rec["MIN_IP_PKT_LEN"], blen)
275             rec["MAX_IP_PKT_LEN"] = max(rec["MAX_IP_PKT_LEN"], blen)

```

Рис. 12. Использование информации о L7-протоколе при формировании потока

После завершения обработки всех пакетов в пределах временного окна выполняется финализация данных. Агрегированные статистики преобразуются в структурированную табличную форму, где каждая строка соответствует одному сетевому потоку, а столбцы содержат

унифицированный набор характеристик соединения. На этом этапе также выполняется расчёт производных метрик и приведение данных к согласованному формату. Формирование итоговой структуры сетевых потоков показано на рис. 13.

```

348 def finalize(self) -> pd.DataFrame:
349     rows = []
350     for _, rec in self.by_flow.items():
351         if rec["_first_ts"] is not None and rec["_last_ts"] is not None:
352             rec["FLOW_DURATION_MILLISECONDS"] = (rec["_last_ts"] - rec["_first_ts"]) * 1000.0
353         if rec["_in_first_ts"] is not None and rec["_in_last_ts"] is not None:
354             rec["DURATION_IN"] = rec["_in_last_ts"] - rec["_in_first_ts"]
355         if rec["_out_first_ts"] is not None and rec["_out_last_ts"] is not None:
356             rec["DURATION_OUT"] = rec["_out_last_ts"] - rec["_out_first_ts"]
357
358         if rec["DURATION_OUT"] > 0:
359             rec["SRC_TO_DST_AVG_THROUGHPUT"] = rec["OUT_BYTES"] / rec["DURATION_OUT"]
360         if rec["DURATION_IN"] > 0:
361             rec["DST_TO_SRC_AVG_THROUGHPUT"] = rec["IN_BYTES"] / rec["DURATION_IN"]
362
363         win = max(rec["DURATION_IN"], rec["DURATION_OUT"], 1e-6)
364         rec["SRC_TO_DST_SECOND_BYTES"] = rec["OUT_BYTES"] / win
365         rec["DST_TO_SRC_SECOND_BYTES"] = rec["IN_BYTES"] / win
366
367         for k in ["MIN_TTL", "MAX_TTL", "SHORTEST_FLOW_PKT", "MIN_IP_PKT_LEN"]:
368             v = rec[k]
369             if v in (np.inf, -np.inf):
370                 rec[k] = np.nan
371
372         rows.append({c: rec.get(c, np.nan) for c in INPUT_COLS})
373     df = pd.DataFrame(rows)
374     df = df.rename(columns={c: _normalize_col_name(c) for c in df.columns})
375     return df

```

Рис. 13. Формирование итоговой структуры сетевых потоков

Полученные структурированные данные являются выходом модуля независимой обработки сетевого трафика и передаются в

модуль предобработки признаков для дальнейшего согласования и анализа.

Модуль обработки журналов регистрации событий и вспомогательных источников

Модуль обработки журналов регистрации событий и вспомогательных источников предназначен для сбора и первичной подготовки событийных данных, отражающих состояние сервисов и компонентов информационной системы. В отличие от модуля независимой обработки сетевого трафика, ориентированного на анализ пакетов и сетевых потоков, данный компонент работает с журналами регистрации и телеметрией вспомогательных средств мониторинга. Его основная задача заключается в формировании унифицированных текстовых представлений событий, используемых в дальнейшем для интерпретации атакующих сценариев, признаки которых не обладают выраженными сетевыми характеристиками.

Общая логика функционирования модуля представлена на рис. 14 и отражает последовательность ключевых этапов обработки событийных данных. Алгоритм включает инициализацию программной среды, задание параметров функционирования, при необходимости — удалённый сбор журналов регистрации по защищённому каналу, а также чтение и объединение локальных журналов по логическим группам источников.

Работа модуля начинается с инициализации и проверки доступности механизма удалённого доступа. В программной реализации предусмотрено условное использование SSH-компонента: при его наличии модуль получает возможность синхронизировать журналы регистрации с удалённых узлов, а при отсутствии автоматически переходит в режим обработки только локальных данных. Такой подход обеспечивает универсальность модуля и его применимость как в распределённых инфраструктурах, так и в изолированных средах. Пример инициализации и проверки доступности SSH-механизма приведён на рис. 15.

После инициализации формируется структура источников событийных данных. Для сохранения смысловой целостности журналов и упрощения последующей

обработки источники группируются по логическим категориям, таким как веб-компоненты, механизмы аутентификации, прикладные сервисы, системные журналы, а также телеметрия IDS и Zeek. Такая группировка позволяет формировать независимые текстовые слои событий и исключает смешивание разнотипных журналов в одном массиве данных. Принцип логической группировки источников показан на рис. 16.



Рис. 14. Блок-схема алгоритма работы модуля обработки журналов регистрации событий и вспомогательных источников

```

13 try:
14     import paramiko # type: ignore
15     HAS_PARAMIKO = True
16     print("[SSH] paramiko успешно импортирован")
17 except Exception as e:
18     print("[SSH] Ошибка импорта paramiko:", e)
19     HAS_PARAMIKO = False

```

Рис. 15. Инициализация модуля обработки журналов регистрации событий и проверка доступности механизма удалённого доступа по SSH

```

21 APP_SOURCES = ['Логи приложений', 'Логи приложений (Web/API/DB/syslog)', 'Логи приложений (дон)']
22 IDS_SOURCES = ['IDS', 'IDS (Suricata/Snort)', 'IDS (дон)']
23 ZEEK_SOURCES = ['Zeek', 'Zeek (DNS/HTTP/SSL/Conn)', 'Zeek (дон)']
24 BEHAV_SOURCES = ['Поведенческие атаки']
25 NET_SOURCES = ['PCAP/NetFlow/IDS', 'PCAP/NetFlow/DNS/TLS/ARP/SNMP', 'PCAP/NetFlow (дон)']

```

Рис. 16. Логическая группировка источников журналов регистрации событий

При необходимости модуль выполняет удалённый сбор журналов регистрации. Для этого используется карта соответствий, определяющая перечень удалённых файлов журналов и их сопоставление локальной структуре каталогов. Данная карта обеспечивает единый формат хранения

данных: журналы, полученные с удалённых узлов, после копирования размещаются в той же структуре, что и локальные файлы, что упрощает дальнейшее чтение и исключает необходимость учитывать происхождение данных при обработке. Пример карты соответствий приведён на рис. 17.

```

33 DEFAULT_REMOTE_LOG_MAP = {
34     "/var/log/auth.log": "auth",
35     "/var/log/secure": "auth",
36     "/var/log/syslog": "system",
37     "/var/log/messages": "system",
38     "/var/log/nginx/access.log": "web",
39     "/var/log/nginx/error.log": "web",
40     "/var/log/apache2/access.log": "web",
41     "/var/log/apache2/error.log": "web",
42     "/var/log/suricata/eve.json": "ids",
43     "/opt/zeek/logs/current/dns.log": "zeek",
44     "/opt/zeek/logs/current/http.log": "zeek",
45     "/opt/zeek/logs/current/ssl.log": "zeek",
46     "/opt/zeek/logs/current/conn.log": "zeek",
47     "/var/log/ufw.log": "firewall",
48 }

```

Рис. 17. Карта соответствий удалённых журналов регистрации и локальной структуры хранения

Непосредственный удалённый сбор реализован в виде последовательности операций, включающих установку SSH-соединения, открытие SFTP-канала и копирование файлов журналов в соответствующие локальные каталоги. В реализации предусмотрены проверки доступности удалённого узла и устойчивое поведение при возникновении ошибок, таких как недоступность части файлов или отсутствие соединения. Реализация процедуры синхронизации журналов по защищённому каналу показана на рис. 18.

В случае если удалённый сбор не требуется, модуль переходит непосредственно к обработке локальных

журналов регистрации.

После подготовки локального набора данных выполняется чтение журналов по логическим группам источников. Для этого используется универсальная процедура чтения файлов, включающая обход каталогов, безопасное открытие файлов и объединение их содержимого в единое текстовое представление. Такой подход не зависит от количества файлов и их разбиения по временным интервалам или службам и позволяет формировать целостный текстовый слой событий, отражающий активность определённого типа. Универсальная процедура чтения журналов показана на рис. 19.

```

85  def _sync_remote_logs(logs_root: str, ssh_hosts: List[Tuple[str, str, str]] | None) -> None:
86  if not ssh_hosts or not HAS_PARAMIKO:
87      return
88  if not os.path.isdir(logs_root):
89      os.makedirs(logs_root, exist_ok=True)
90  for host, user, password in ssh_hosts:
91  try:
92      client = paramiko.SSHClient()
93      client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
94      client.connect(hostname=host, username=user, password=password, timeout=5)
95      sftp = client.open_sftp()
96  except Exception as e:
97      print(f"[SSH] Не удалось подключиться к {host}: {e}")
98      try:
99          client.close()
100     except Exception:
101         pass
102     continue
103  try:
104      for rpath, subdir in DEFAULT_REMOTE_LOG_MAP.items():
105      try:
106          dest_dir = os.path.join(logs_root, subdir, host)
107          os.makedirs(dest_dir, exist_ok=True)
108          local_path = os.path.join(dest_dir, os.path.basename(rpath))
109          sftp.get(rpath, local_path)
110     except FileNotFoundError:
111         continue
112     except Exception as e:
113         print(f"[SSH] Ошибка копирования {rpath} в {host}: {e}")
114         continue
115  finally:
116      try:
117          sftp.close()
118     except Exception:
119         pass
120      try:
121          client.close()
122     except Exception:
123         pass
124

```

Рис. 18. Удалённый сбор журналов регистрации событий по защищённому каналу SSH и копирование файлов в локальную структуру

```

50  def _read_all(root: str) -> str:
51      chunks = []
52      if not root or not os.path.isdir(root):
53          return ""
54      for r, _, files in os.walk(root):
55          for name in files:
56              path = os.path.join(r, name)
57              try:
58                  with open(path, "r", encoding="utf-8", errors="ignore") as f:
59                      chunks.append(f.read())
60              except Exception:
61                  continue
62      return "\n".join(chunks)

```

Рис. 19. Универсальная процедура чтения файлов журналов и формирования текстового слоя событий

Для повышения удобства обращения к источникам данных используются функции-обёртки, каждая из которых формирует путь к соответствующей группе журналов и вызывает универсальную процедуру чтения. Благодаря этому архитектура модуля остаётся структурированной, а добавление

новых источников событийных данных сводится к расширению перечня групп и определению соответствующей функции-обёртки. Пример набора функций чтения журналов по логическим группам источников приведён на рис. 20.

```

64 def read_web_logs(logs_root: str) -> str:
65     return _read_all(os.path.join(logs_root, "web"))
66
67 def read_auth_logs(logs_root: str) -> str:
68     return _read_all(os.path.join(logs_root, "auth"))
69
70 def read_app_logs(logs_root: str) -> str:
71     return _read_all(os.path.join(logs_root, "app"))
72
73 def read_ids_logs(logs_root: str) -> str:
74     return _read_all(os.path.join(logs_root, "ids"))
75
76 def read_firewall_logs(logs_root: str) -> str:
77     return _read_all(os.path.join(logs_root, "firewall"))
78
79 def read_system_logs(logs_root: str) -> str:
80     return _read_all(os.path.join(logs_root, "system"))
81
82 def read_zeek_logs(logs_root: str) -> str:
83     return _read_all(os.path.join(logs_root, "zeek"))

```

Рис. 20. Функции чтения журналов регистрации по логическим группам источников

Работа модуля организована в циклическом режиме. В каждом цикле обновляется набор событийных данных, при необходимости выполняется удалённая синхронизация журналов, после чего выдерживается заданный интервал и начинается следующий проход обработки. Начало цикла обновления и условный вызов процедуры удалённого сбора показаны на рис. 21, а механизм выдерживания временного интервала между циклами обработки — на рис. 22.

```

233 def log_detection_loop(logs_root: str, interval: float = 10.0, ssh_hosts: List[Tuple[str, str, str]] | None = None):
234     if not logs_root:
235         return
236     os.makedirs(logs_root, exist_ok=True)
237     print(f"Запущен фоновый анализ логов в {logs_root}")
238     if ssh_hosts and not HAS_PARAMIKO:
239         print("paramiko не установлен, сбор логов по SSH будет пропущен.")
240     while True:
241         try:
242             _sync_remote_logs(logs_root, ssh_hosts)

```

Рис. 21. Циклическое обновление событийных данных и вызов процедуры удалённой синхронизации журналов

```

252         time.sleep(interval)

```

Рис. 22. Выдерживание заданного интервала между циклами обработки журналов регистрации

Такой режим функционирования обеспечивает регулярное обновление событийных данных и поддержание их актуальности без накопления избыточных объёмов информации за один цикл.

Сетевые данные, сформированные в модуле независимой обработки сетевого трафика, и событийные данные, подготовленные в рамках данного модуля, существенно различаются по структуре и формату представления. Для их совместного использования требуется согласование результатов и приведение данных к унифицированному виду. Данная задача решается в модуле предобработки признаков, рассматриваемом в следующем разделе.

Модуль предобработки выявленных признаков

Модуль предобработки выявленных признаков является связующим элементом конвейера обработки данных. Он принимает результаты работы двух независимых подсистем — структурированные сетевые потоки, сформированные модулем независимой обработки сетевого трафика, и событийные данные, подготовленные модулем обработки журналов регистрации событий, — и приводит их к согласованному виду, удобному для последующих этапов анализа. Принципиально важно, что на данном этапе не выполняется формирование итоговых выводов и не принимаются

решения о наличии атак: задача модуля ограничивается проверкой входных данных, устранением структурных несоответствий и преобразованием признаков к единому формату.

Общая логика функционирования модуля предобработки представлена на рис. 23 и отражает последовательность основных действий, включающую приём входных данных, их валидацию, приведение сетевых и событийных представлений к согласованной структуре, предварительное преобразование признаков и формирование унифицированного пакета данных для последующего использования.

Реализация предобработки в системе опирается на принцип фиксированного признакового пространства. Для сетевой части это означает, что каждая итоговая запись потока должна содержать строго определённый набор полей, соответствующий входу последующих процедур анализа. Такой набор признаков задаётся в виде списка колонок и используется в качестве эталонной структуры данных. Формирование модельного признакового пространства и нормализация имён входных полей показаны на рис. 24.

После того как модуль обработки сетевого трафика сформировал таблицу потоков за очередное временное окно, предобработка начинается с валидации структуры данных и приведения типов. На практике это включает две ключевые операции: при необходимости добавление отсутствующих полей из эталонного списка признаков и принудительное приведение значений колонок к числовому виду с безопасной обработкой ошибок преобразования. Такой подход защищает конвейер от неполных данных и ситуаций, когда отдельные признаки не были извлечены из трафика, например из-за особенностей протоколов или отсутствия соответствующих полей. Реализация заполнения недостающих признаков и типизации данных приведена на рис. 25.

Следующим этапом выполняется предварительное преобразование признаков, которое реализуется путём применения сохранённого препроцессора, который обеспечивает одинаковую обработку данных

как на этапе эксплуатации, так и на этапе обучения модели.



Рис. 23. Блок-схема алгоритма работы модуля предобработки выявленных признаков


```

43 INPUT_COLS = [
44     "L4_DST_PORT", "PROTOCOL", "L7_PROTO", "IN_BYTES", "IN_PKTS", "OUT_BYTES", "OUT_PKTS",
45     "TCP_FLAGS", "CLIENT_TCP_FLAGS", "SERVER_TCP_FLAGS", "FLOW_DURATION_MILLISECONDS",
46     "DURATION_IN", "DURATION_OUT", "MIN_TTL", "MAX_TTL", "LONGEST_FLOW_PKT", "SHORTEST_FLOW_PKT",
47     "MIN_IP_PKT_LEN", "MAX_IP_PKT_LEN", "SRC_TO_DST_SECOND_BYTES", "DST_TO_SRC_SECOND_BYTES",
48     "RETRANSMITTED_IN_BYTES", "RETRANSMITTED_IN_PKTS", "RETRANSMITTED_OUT_BYTES", "RETRANSMITTED_OUT_PKTS",
49     "SRC_TO_DST_AVG_THROUGHPUT", "DST_TO_SRC_AVG_THROUGHPUT", "NUM_PKTS_UP_TO_128_BYTES",
50     "NUM_PKTS_128_TO_256_BYTES", "NUM_PKTS_256_TO_512_BYTES", "NUM_PKTS_512_TO_1024_BYTES",
51     "NUM_PKTS_1024_TO_1514_BYTES", "TCP_WIN_MAX_IN", "TCP_WIN_MAX_OUT", "ICMP_TYPE", "ICMP_IPV4_TYPE",
52     "DNS_QUERY_ID", "DNS_QUERY_TYPE", "DNS_TTL_ANSWER", "FTP_COMMAND_RET_CODE",
53 ]

```

Рис. 24. Определение фиксированного признакового пространства для предобработки сетевых данных

```

384 def predict_batch(model, preproc, encoders, df: pd.DataFrame):
385     for c in INPUT_COLS:
386         if c not in df.columns:
387             df[c] = np.nan
388             df[c] = pd.to_numeric(df[c], errors="coerce")

```

Рис. 25. Валидация структуры сетевых данных: добавление отсутствующих признаков и приведение типов

В частности, такой препроцессор может включать масштабирование, нормализацию, обработку пропусков и иные преобразования, зафиксированные при подготовке модели.

Использование единого сохранённого механизма преобразования принципиально важно для воспроизводимости результатов и предотвращения расхождений между обучающими и эксплуатационными данными. Применение препроцессора к сформированному вектору признаков показано на рис. 26.

```

389 Xp = preproc.transform(df[INPUT_COLS])

```

Рис. 26. Применение сохранённого препроцессора для предварительного преобразования признаков

```

521 df = agg.finalize()
522 if df.empty:
523     print("...нет собранных потоков за окно.")
524     continue

```

Рис. 27. Контроль корректности входных данных окна: обработка ситуации отсутствия сформированных потоков

Событийная часть данных, формируемая модулем обработки журналов регистрации и вспомогательных источников, поступает в систему параллельно в виде подготовленных текстовых слоёв по логическим группам источников. В текущей архитектуре сбор и обновление событийных данных запускаются в фоновом потоке и не блокируют обработку

Отдельное внимание в модуле уделяется контролю корректности входных данных по завершении обработки очередного временного окна. Перед переходом к следующим этапам система проверяет наличие сформированных сетевых потоков. Если за рассматриваемый интервал трафика потоки не были сформированы, дальнейшая обработка для данного окна не выполняется. Это позволяет избежать ложных действий на пустых входных данных и повышает устойчивость системы при работе в режиме, близком к реальному времени. Логика проверки и обработки ситуации отсутствия потоков представлена на рис. 27.

Сетевой трафик. Такое решение позволяет поддерживать непрерывность работы системы: сетевые потоки обрабатываются в оконном режиме, а журналы обновляются независимо с заданным интервалом. Точка интеграции событийных данных и запуск фоновой компоненты обновления журналов регистрации показаны на рис. 28.

```

456  |   if args.logs_dir:
457  |       th = threading.Thread(
458  |           target=log_detection_loop,
459  |           args=(args.logs_dir, 10.0, ssh_hosts),
460  |           daemon=True,
461  |       )
462  |       th.start()

```

Рис. 28. Точка интеграции событийных данных: запуск фонового обновления журналов регистрации

После согласования форматов, валидации структуры и предварительного преобразования признаков формируется унифицированное представление данных, пригодное для совместной интерпретации результатов, полученных из различных источников наблюдения. На следующем этапе система выполняет объединение этих результатов и формирование итоговой картины по текущему временному окну, что реализуется в модуле принятия решений.

Модуль принятия решений

Модуль принятия решений является завершающим компонентом разработанной системы и предназначен для формирования итоговой интерпретации результатов анализа за текущее временное окно наблюдения. На вход данного модуля поступают результаты обработки, полученные из двух независимых источников: анализа сетевого трафика и анализа журналов регистрации событий и вспомогательных источников. Основная задача модуля заключается в объединении этих результатов, устранении дублирования и формировании итогового перечня выявленных шаблонов атак в терминах классификатора CAPEC [18].

Общая логика функционирования модуля принятия решений представлена на рис. 29 и 30. Заканчивается формированием итогового отчёта и регистрацией результатов для оператора системы.

Работа модуля начинается с подготовки программного окружения и запуска вспомогательных компонентов. В частности, при наличии конфигурации для анализа журналов регистрации событий инициируется фоновый поток, обеспечивающий независимый сбор и обработку логов. Такое решение позволяет выполнять событийный анализ параллельно с обработкой сетевого трафика и не

блокировать основной цикл работы системы. Запуск фонового анализа журналов регистрации событий показан на рис. 31.

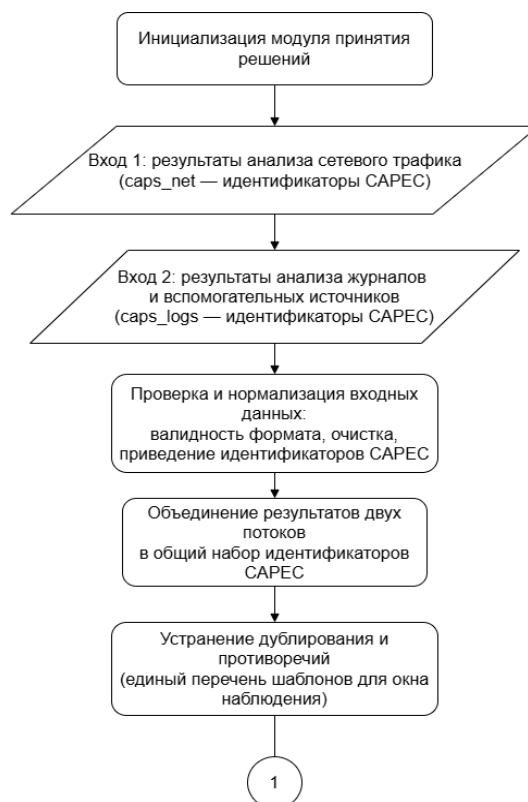


Рис. 29. Блок-схема алгоритма работы модуля принятия решений и формирования итоговых результатов

После инициализации вспомогательных компонентов в рамках основного цикла обработки формируются результаты анализа сетевого трафика. Для каждого временного окна система обрабатывает результаты классификации сетевых потоков и извлекает идентификаторы шаблонов CAPEC [18], полученные по итогам работы модели. Таким образом формируется множество паттернов, соответствующих атакующим сценариям, признаки которых были выявлены на основе сетевого трафика. Процесс формирования данного множества показан на рис. 32.



Рис. 30. Блок-схема алгоритма работы модуля принятия решений и формирования итоговых результатов

```

456 | if args.logs_dir:
457 |     th = threading.Thread(
458 |         target=log_detection_loop,
459 |         args=(args.logs_dir, 10.0, ssh_hosts),
460 |         daemon=True,
461 |     )
462 |     th.start()

```

Рис. 31. Запуск фонового анализа журналов регистрации событий

```

528 | ml_capec_detected = set()
529 |
530 | for i, row in df.iterrows():
531 |     flow_id = f"{int(row.get('PROTOCOL',0))}/{int(row.get('L4_DST_PORT',0))}"
532 |     print(f"\n[FLOW #{i}] proto/port={flow_id} OUT/IN={row.get('OUT_BYTES',0)} / {row.get('IN_BYTES',0)} dur_ms={row.get('FLOW_DURATION_MILLISECONDS',0):.1f}")
533 |     for head in OUTPUT_COLS:
534 |         if head not in preds[i]:
535 |             continue
536 |         label, prob = preds[i][head]
537 |         print(f"  {head}: {label} ({prob:.3f})")
538 |         ml_capec_detected.add(label)

```

Рис. 32. Формирование набора идентификаторов CAPEC по результатам анализа сетевого трафика

Сформированный набор идентификаторов CAPEC [18], полученный по результатам анализа сетевого трафика, используется для регистрации и вывода промежуточного результата анализа. На этом

этапе система фиксирует перечень обнаруженных шаблонов атак и обеспечивает их доступность для последующего объединения с результатами событийного анализа. Логика вывода представлена на рис. 33.

```

540 | if ml_capec_detected:
541 |     print("\n[ML] Обнаружены CAPEC (по сетевому трафику, модель):")
542 |     for c in sorted(ml_capec_detected):
543 |         print(f" - {c}")

```

Рис. 33. Вывод идентификаторов CAPEC, обнаруженных по результатам анализа сетевого трафика

Параллельно с анализом сетевого трафика в системе формируются результаты анализа журналов регистрации событий и вспомогательных источников. Обработка логов выполняется в отдельном компоненте и приводит к формированию множества шаблонов CAPEC [18], выявленных на основе событийных данных. Эти результаты

отражают атакующие сценарии, признаки которых проявляются преимущественно на уровне операционных систем, прикладных сервисов и телеметрии средств мониторинга. Формирование множества идентификаторов CAPEC [18] по результатам событийного анализа показано на рис. 34.

```

217 | def detect_all_from_logs(logs_root: str) -> Set[str]:
218 |     all_detected: Set[str] = set()
219 |     web_app_text = "\n".join([read_web_logs(logs_root),
220 |                               read_app_logs(logs_root),
221 |                               read_auth_logs(logs_root)])
222 |     all_detected |= detect_from_web_app(web_app_text)
223 |     ids_text = read_ids_logs(logs_root)
224 |     all_detected |= detect_from_ids(ids_text)
225 |     zeek_text = read_zeek_logs(logs_root)
226 |     all_detected |= detect_from_zeek(zeek_text)
227 |     fw_text = read_firewall_logs(logs_root)
228 |     all_detected |= detect_from_firewall(fw_text)
229 |     sys_text = read_system_logs(logs_root)
230 |     all_detected |= detect_from_system(sys_text)
231 |     return all_detected

```

Рис. 34. Формирование набора идентификаторов CAPEC по результатам анализа журналов регистрации событий

По завершении очередного цикла анализа журналов регистрации выполняется вывод результатов событийного анализа. В зависимости от наличия или отсутствия выявленных шаблонов CAPEC [18] система либо регистрирует их перечень, либо фиксирует отсутствие совпадений для

текущего цикла наблюдения. Такая логика позволяет корректно учитывать как подтверждённые атакующие сценарии, так и отсутствие признаков атак в анализируемом интервале. Соответствующий фрагмент программной реализации приведён на рис. 35.

```

245 | print("\n[LOG-ANALYZER] Обнаружены шаблоны CAPEC по логам и SSH-собранным данным:")
246 | for c in sorted(caps):
247 |     print(f" - {c}")
248 | else:
249 |     print("\n[LOG-ANALYZER] CAPEC по логам в этом цикле не обнаружены.")

```

Рис. 35. Вывод результатов анализа журналов регистрации событий за текущий цикл

На завершающем этапе работы модуля результаты анализа сетевого трафика и журналов регистрации событий используются для формирования итоговой картины происходящих атакующих воздействий за текущее временное окно. Устранение дублирования и агрегирование идентификаторов CAPEC [18] из независимых источников обеспечивают

целостное и интерпретируемое представление о состоянии безопасности системы, которое может быть использовано для регистрации инцидентов и информирования оператора.

Демонстрация работы программных модулей

Для демонстрации функционирования разработанных программных модулей в локальной вычислительной сети использовался основной исполняемый модуль системы, обеспечивающий параллельную обработку сетевого трафика, журналов регистрации событий и данных от вспомогательных средств мониторинга. Запуск системы осуществляется вручную из консоли операционной системы с указанием параметров, определяющих режим её работы, включая сетевой интерфейс захвата, временной интервал обработки данных, расположение артефактов обученной модели, директорию журналов регистрации и параметры удалённого узла для синхронизации логов по защищённому каналу.

Заданные параметры определяют ключевые элементы функционирования комплекса: сетевой интерфейс, с которого производится захват пакетов, интервал анализа данных, каталог с артефактами

обученной модели, локальное хранилище журналов регистрации событий, а также удалённый узел, с которого осуществляется извлечение логов по протоколу SSH. После запуска система переходит в непрерывный режим работы и одновременно анализирует сетевой трафик и событийные данные, дополняя их информацией от вспомогательных инструментов мониторинга, включая PCAP-захват, NetFlow/IPFIX-телеметрию, журналы Zeek прикладного уровня и сигнатурные данные IDS Suricata/Snort. Указанные средства используются не как самостоятельные классификаторы, а как дополнительные источники наблюдения, расширяющие контекст анализа и повышающие полноту выявляемых признаков.

Работа системы в режиме, близком к реальному времени, проиллюстрирована на рис. 36, где представлен фрагмент консольного вывода в момент проведения тестового сканирования портов с другого устройства в сети.

```

mance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-12-06 19:34:28.894637: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
[SSH] paramiko успешно импортирован
Запущен фоновый анализ логов в /mnt/data/test
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1765849669.702041 1667 cuda_executor.cc:1309] INTERNAL: CUDA Runtime error: Failed call to cudaGetRuntimeVersion: Error loading CUDA libraries. GPU will not be used.: Error loading CUDA libraries. GPU will not be used.
W0000 00:00:1765849669.709956 1667 gpu_device.cc:2342] Cannot dlopen some GPU libraries. Please make sure the missing libraries mentioned above are installed properly if you would like to use GPU. Follow the guide at https://www.tensorflow.org/install/gpu for how to download and setup the required libraries for your platform.
Skipping registering GPU devices...
Accel: CPU, AMP=True
▶ Старт DPI-инференса. Ctrl+C – остановка.

[LOG-ANALYZER] CAPEC по логам в этом цикле не обнаружены.

[ML] Обнаружены CAPEC-ID (по сетевому трафику, модель):
- CAPEC-300
...нет пакетов за окно.
...нет собранных потоков за окно.
...нет пакетов за окно.

[LOG-ANALYZER] CAPEC по логам в этом цикле не обнаружены.

[ML] CAPEC по трафику с порогом уверенности 0.85 не обнаружены.
...нет пакетов за окно.
...нет пакетов за окно.
...нет пакетов за окно.
...нет пакетов за окно.

[LOG-ANALYZER] CAPEC по логам в этом цикле не обнаружены.

[ML] CAPEC по трафику с порогом уверенности 0.85 не обнаружены.
...нет пакетов за окно.
...нет пакетов за окно.
...нет пакетов за окно.
...нет пакетов за окно.

[LOG-ANALYZER] CAPEC по логам в этом цикле не обнаружены.

[ML] CAPEC по трафику с порогом уверенности 0.85 не обнаружены.

```

Рис. 36. Детектирование CAPEC-300

В ходе атаки система фиксирует рост числа попыток установления соединений с различными портами целевого узла и формирует признаки наблюдаемых потоков, опираясь не только на анализ сетевых

пакетов, но и на сведения от Zeek и NetFlow/IPFIX. Эти источники дополняют информацию о типе взаимодействий, частоте соединений и характере отклонённых запросов. После преобразования признаков

классификатор сопоставляет наблюдаемое поведение с формализованными шаблонами CAPEC [18], в результате чего в консоли появляется сообщение о выявленной атаке с указанием её идентификатора. Данная демонстрация подтверждает, что модуль обработки сетевого трафика корректно реагирует на аномальные модели поведения и

способен распознавать атакующие действия, обладающие выраженными сетевыми признаками.

Дополнительная проверка функционирования системы представлена на рис. 37, где показан результат обработки тестового журнала регистрации событий.



Рис. 37. Обработка тестового журнала регистрации событий

В данном случае система выявляет признаки нескольких шаблонов атак CAPEC [18] на основе анализа лог-файлов и данных, полученных с удалённого узла по защищённому каналу. Каждый из обнаруженных идентификаторов соответствует определённому классу угроз и отражает конкретный тип атакующего поведения. Характерно, что набор выявленных шаблонов включает взаимодополняющие категории, охватывающие как атаки, связанные с обходом механизмов аутентификации, так и действия, указывающие на попытки эксплуатации уязвимостей или внесения несанкционированных изменений в систему. Это демонстрирует способность событийного модуля корректно обрабатывать разнородные журнальные записи, выделять ключевые признаки и сопоставлять их с формализованными моделями атак.

В совокупности результаты, представленные на рис. 36 и 37, подтверждают согласованную работу всех компонентов разработанной системы. Модуль обработки сетевого трафика обеспечивает своевременное выявление атак, проявляющихся на уровне сетевых взаимодействий, тогда как модуль анализа журналов регистрации позволяет обнаруживать сценарии, признаки которых фиксируются преимущественно в событиях

операционных систем и прикладных сервисов. Использование вспомогательных источников мониторинга дополняет наблюдаемые данные и формирует более целостную картину происходящих процессов, что особенно важно при диагностике многоэтапных и скрытых атакующих сценариев.

Заключение

В статье разработана и реализована архитектура системы автоматизированного обнаружения и классификации кибератак, основанная на независимой и параллельной обработке сетевого трафика и журналов регистрации событий. Предложенный подход позволяет учитывать разнородные проявления атакующих сценариев, возникающие на сетевом и событийном уровнях, и формировать целостную интерпретируемую картину инцидентов информационной безопасности.

В рамках работы спроектированы и программно реализованы ключевые модули системы, обеспечивающие сбор и агрегацию сетевых потоков, обработку журналов регистрации и вспомогательных источников мониторинга, предобработку выявленных признаков и формирование итоговых решений. Разграничение функциональных ролей между модулями и использование формализованных интерфейсов

обеспечивают масштабируемость системы и возможность расширения набора детектируемых шаблонов атак без изменения базовой логики работы.

Особое внимание уделено модулю принятия решений, выполняющему агрегирование результатов анализа, полученных из независимых источников, и их сопоставление с шаблонами атак классификатора CAPEC [18]. Такой подход позволяет устранить дублирование результатов, повысить интерпретируемость выявляемых атак и обеспечить согласованную классификацию атакующих сценариев в терминах формализованной таксономии угроз.

Проведённая демонстрация работы системы в условиях локальной сети подтвердила корректность функционирования разработанных программных модулей. Экспериментальные примеры показали способность системы выявлять как сетевые атаки с выраженными аномальными характеристиками, так и сценарии, проявляющиеся преимущественно в журналах регистрации событий и вспомогательной телеметрии. Использование дополнительных источников мониторинга позволило расширить контекст анализа и повысить полноту обнаружения многоэтапных и скрытых атак.

Полученные результаты подтверждают практическую применимость разработанного программного комплекса для использования в корпоративных и распределённых сетевых инфраструктурах в качестве основы для систем мониторинга и реагирования на инциденты информационной безопасности. Разработанная архитектура и реализованные программные модули создают основу для дальнейшего развития системы, включая интеграцию обучаемых моделей классификации и проведение углублённой оценки эффективности обнаружения атак, что является предметом последующих исследований.

Список литературы

1. Scarfone K., Mell P. Guide to Intrusion Detection and Prevention Systems (IDPS). NIST Special Publication 800-94. National Institute of

Standards and Technology, Gaithersburg, MD, 2007.

2. Roesch M. Snort: Lightweight Intrusion Detection for Networks // Proceedings of the 13th USENIX Conference on System Administration (LISA'99). Seattle, WA, 1999. P. 229–238.

3. Open Information Security Foundation. Suricata User Guide. URL: <https://suricata.io> (дата обращения: 08.09.25).

4. Paxson V. Bro: A System for Detecting Network Intruders in Real-Time // Computer Networks. 1999. Vol. 31, No. 23–24. P. 2435–2463.

5. Cisco Systems. Cisco Secure Intrusion Detection System: Architecture and Configuration Guide. URL: <https://www.cisco.com> (дата обращения: 08.09.25).

6. Fortinet. FortiGate IPS Administration Guide. URL: <https://docs.fortinet.com> (дата обращения: 08.09.25).

7. Wazuh Inc. Wazuh Documentation. URL: <https://documentation.wazuh.com> (дата обращения: 08.09.25).

8. Trend Micro (OSSEC Project). OSSEC Documentation. URL: <https://www.ossec.net> (дата обращения: 08.09.25).

9. Kaspersky Lab. Endpoint Detection and Response: Technical Overview. URL: <https://www.kaspersky.com> (дата обращения: 08.09.25).

10. CrowdStrike. CrowdStrike Falcon Platform: Technical Documentation. URL: <https://www.crowdstrike.com> (дата обращения: 08.09.25).

11. Microsoft. Microsoft Defender for Endpoint Documentation. URL: <https://learn.microsoft.com> (дата обращения: 08.09.25).

12. Splunk Inc. Splunk Enterprise Security Architecture Overview URL: <https://www.splunk.com> (дата обращения: 08.09.25).

13. IBM Corporation. IBM QRadar SIEM: Architecture and Deployment Guide. URL: <https://www.ibm.com> (дата обращения: 08.09.25).

14. OpenText (Micro Focus). ArcSight SIEM Platform Overview. URL: <https://www.microfocus.com> (дата обращения: 08.09.25).

15. Elastic. Elastic Security Description. URL: <https://www.ptsecurity.com>
Documentation. URL: <https://www.elastic.co> (дата обращения: 08.09.25).
16. Graylog. Graylog Security and SIEM Use Cases. URL: <https://www.graylog.org> (дата обращения: 08.09.25).
17. Positive Technologies. PT Network Attack Discovery (PT NAD): Product
18. MITRE Corporation. CAPEC — Common Attack Pattern Enumeration and Classification. URL: <https://capec.mitre.org> (дата обращения: 08.09.25).

Финансовый университет при Правительстве Российской Федерации
Financial University under the Government of the Russian Federation

Воронежский государственный технический университет
Voronezh State Technical University

Поступила в редакцию 22.11.2025

Информация об авторах

Васильченко Алексей Павлович – аспирант, Финансовый университет при Правительстве Российской Федерации, e-mail: rainichek@yandex.ru

Попова Елена Андреевна – студент, Воронежский государственный технический университет, e-mail: elpov0211@gmail.com

Жуков Никита Павлович – аспирант, Воронежский государственный технический университет, e-mail: znp8b00ff@gmail.com

Дешина Анна Евгеньевна – канд. техн. наук, доцент, Воронежский государственный технический университет, e-mail: alexanderostapenkoias@gmail.com

CYBER-ATTACK DETECTION AND CLASSIFICATION BASED ON COMPREHENSIVE ANALYSIS OF NETWORK TRAFFIC AND EVENT LOGS: A METHODOLOGY FOR GENERATING TRAINING DATA AND A SET OF DETECTED PATTERNS

A.P. Vasilchenko, E.A. Popova, N.P. Zhukov, A.E. Deshina

The article discusses the task of generating training data for automated detection and classification systems for cyber attacks based on a comprehensive analysis of network traffic and event logs. A method for selecting and structuring detectable attack patterns using the CAPEC taxonomy is proposed, providing formalization of data markup processes and defining the boundaries of the system's applicability. The differentiation of CAPEC templates into machine learning models used for training and those identified by software methods of event data analysis is substantiated. It is shown that the generated set of detectable patterns covers about 40% of the CAPEC list and allows you to create a reproducible training set focused on interpretable and scenario-oriented detection of cyber attacks.

Keywords: attack detection, attack classification, network traffic, event logs, CAPEC, training data.

Submitted 22.11.2025

Information about the authors

Alexey P. Vasilchenko – graduate student, Financial University under the Government of the Russian Federation, e-mail: rainichek@yandex.ru

Elena A. Popova – student, Voronezh State Technical University, e-mail: elpov0211@gmail.com

Nikita P. Zhukov – graduate student, Voronezh State Technical University, e-mail: znp8b00ff@gmail.com

Anna E. Deshina – Cand. Sc. (Technical), Associated Professor, Voronezh State Technical University, e-mail: alexanderostapenkoias@gmail.com